INKLAB

Advanced Open-Source Heterogeneous Development Platform

Revision: v2 Design Phase | Status: Active Development | Hardware: STM32H562 + iCE40UP5K + E-Paper Display

1. Project Origin & Vision

1.1 Where InkLab Comes From

InkLab originated in a Mechatronics senior design course with the initial goal of building a non-invasive, continuous glucose monitoring device. The team designed a compact wearable (PCB de Control + PCB SensorHub stacked architecture) integrating NIR spectroscopy (850–1500 nm), dual SpO_2, IR thermometry, and ambient sensors — all processed through a custom MCU+FPGA platform.

A patentability analysis was conducted, revealing that while commercial non-invasive glucose devices exist (validating the technology), the space is relatively open in Mexico and Latin America — few patents, and none combining the hardware-software integration approach the team was pursuing. The core insight: the problem is not the sensing physics, it is the **difficulty of integrating hardware complexity with software algorithms**, particularly for researchers coming from domains like signal processing, AI/ML, or medicine who are not embedded engineers.

Faced with the timeline and resource constraints of a semester project, the team made a deliberate and technically sound pivot: rather than shipping an incomplete glucose monitor, **they identified the harder and more general problem** — and built a solution to that instead. InkLab is the result.

1.2 The Problem InkLab Solves

Embedded systems complexity is a major barrier for researchers from adjacent fields. A biomechanics researcher with a novel IMU algorithm should not need to understand DMA buffer management. A medical AI team validating a glucose model should not spend months on MCU bringup. Currently, the typical choices are:

- **Underpower:** Use a simple microcontroller (e.g., Arduino) that is too limited for the algorithm, forcing premature compromises.
- **Overkill:** Use a Linux SBC (e.g., Raspberry Pi) just to read sensor data, adding cost, power consumption, boot time, and complexity that is not warranted.
- **Long custom development:** Build a custom embedded platform from scratch — which requires embedded expertise the researcher does not have.

InkLab eliminates this gap by providing a **programmable, open-source, heterogeneous platform** that a researcher can deploy on day one through a web interface, without touching a compiler or a datasheet.

2. System Architecture Overview

InkLab is organized into four cooperating layers, each decoupling a different dimension of complexity:

Layer	Role
Layer 1: Hardware Acceleration	iCE40UP5K FPGA — parallel, deterministic, real-time. Handles e-paper display timing, SWD signal generation, custom protocols, DSP.
Layer 2: RTOS Orchestration	STM32H562 MCU running FreeRTOS (→ Zephyr). System state, power management, task scheduling, USB communication.
Layer 3: Vue Web Interface	Browser-based DevConsole connecting over USB-CDC. Config, Debug, and App modes. The primary interaction layer for researchers.
Layer 4: Code Sandboxes	SD card macro interpreter (JavaScript-like scripts) and planned WAMR (WebAssembly) runtime. Allows logic execution without reflashing.

3. Primary MCU: STM32H562 (v2)

3.1 Why the MCU Was Upgraded: G0B1 → H562

Version 1 of InkLab used the **STM32G0B1CEUT6** (Cortex-M0+, 64 MHz, 256 KB Flash, 144 KB RAM). This choice was appropriate for the initial prototype given its low cost, small package, and low power characteristics. However, as the software stack matured — adding FreeRTOS, the frontend API, FATFS, FPGA management, power monitoring, and now WAMR — several hard limitations emerged that drove the upgrade to the **STM32H562RIV** (Cortex-M33, 250 MHz, 2 MB Flash, 640 KB RAM).

Parameter	STM32G0B1 (v1)	STM32H562 (v2)
CPU Core	Cortex-M0+ (ARMv6-M)	Cortex-M33 (ARMv8-M + TrustZone)
Clock Speed	64 MHz	250 MHz
Flash	256 KB (single bank)	2 MB (dual bank — live OTA)
RAM	144 KB SRAM	640 KB SRAM
FPU	None — SW float only	Hardware FPU — real DSP performance
DSP extensions	None	Full DSP instruction set (SIMD, MAC)

Parameter	STM32G0B1 (v1)	STM32H562 (v2)
USB	USB FS (shared)	USB FS dedicated peripheral
IoT / Security	None	TrustZone, Secure Boot, crypto accelerators
WAMR headroom	Insufficient — platform limit	Tight but feasible (~150KB of 640KB)
Debug	SWD only, no ETM	SWD + ETM trace support

The key upgrade drivers were: (1) **heap headroom** — 144 KB was exhausted by FreeRTOS + FATFS + USB buffers alone, leaving no room for WAMR; (2) **Cortex-M33** — the M0+ lacks DSP extensions and an FPU, making it inadequate for any signal processing on the device; (3) **IoT readiness** — TrustZone and hardware cryptography are prerequisites for secure remote deployment; (4) **dual-bank Flash** — enabling live OTA firmware updates without downtime; and (5) **platform ceiling** — the G0B1 imposed hard limits that would have required a board respin anyway once more subsystems were added.

4. FPGA Co-Processor: Lattice iCE40UP5K

4.1 Role and Capabilities

The **Lattice iCE40UP5K** handles everything that requires real-time determinism, parallelism, or hardware-level signal timing that would be impossible or prohibitively expensive on the MCU. With 5,280 LUTs, 8 DSP blocks, and 128 Kbit of BRAM, it is synthesized using the fully open-source **OSS CAD Suite (Yosys + nextpnr + IceStorm)** toolchain — no proprietary vendor tools required.

The current synthesis shows only 10% LUT utilization and 40% I/O utilization, confirming substantial headroom for future RTL modules. Max clock frequency is 46.25 MHz (operating at 32 MHz in the current design).

4.2 E-Paper Display Pipeline

The most visually distinctive feature of InkLab is its **e-paper (EPD) display**, driven through a parallel 8-bit interface from the FPGA. The MCU generates images using LVGL or a compatible 1bpp rendering library, then transmits the framebuffer to the FPGA over SPI. The FPGA handles the precise, timing-critical parallel waveform sequencing required by the e-paper controller.

This architecture achieves **10–30 FPS** on an e-paper display — far beyond what typical e-paper drivers support — making the UI feel genuinely responsive. A critical RTL feature is **overdrive protection**: the FPGA compares each incoming frame against the previous one and suppresses driving on pixels that have not changed. This reduces unnecessary electrode stress, significantly extending display lifetime.

The display power management is handled by a **TPS65185** E-Ink PMIC, which generates the multiple high-voltage rails required by the EPD panel (+22V, +15V, -20V, -15V, -3.15V, and a dedicated 3V3_EPD) from the main 3.3V bus.

4.3 Bitstream Slot System

The FPGA is configured from bitstream images stored on the SD card. The system is currently configured for **16 named slots**, each with an associated SPI clock divider. This number is a practical default chosen by the team — it is a compile-time constant that any developer can trivially change in the firmware to suit their needs. Slot switching is live at runtime; the MCU manages the configuration handshake without power cycling the FPGA.

4.4 Embedded SWD Debug Core (BMP Wrapper)

One of the most technically innovative elements of InkLab is its embedded debug capability. Rather than requiring an external JTAG/SWD probe, the board generates SWD signals itself through a three-layer architecture:

1. **MCU Layer — RTOS-compatible BMP wrapper:** A modified version of the Black Magic Probe firmware runs as a FreeRTOS task on the STM32H562. The ADIv5 transport layer is patched to route debug commands over SPI to the FPGA instead of directly toggling GPIO pins.
2. **FPGA Layer — SWD RTL module (already implemented):** A custom Verilog module receives SWD command packets from the MCU over SPI and handles the precise hardware-level signal toggling of SWDIO and SWDCLK. JTAG could be added in the future if the use case warrants it.
3. **Result — High-speed parallel programming:** This architecture targets at least 100 KB/s programming throughput, making firmware uploads seamless. Additionally, because the FPGA handles the physical layer, it is possible to bit-bang multiple SWD targets simultaneously from a single set of SPI commands — allowing one InkLab board to program several daughterboard devices in parallel.

Additional advantages include: no Windows driver headaches (it appears as a standard USB CDC device), GDB server functionality for live debugging, and the ability to operate the debug core as just another FPGA bitstream slot with its own clock configuration.

5. Power Architecture

5.1 Overview

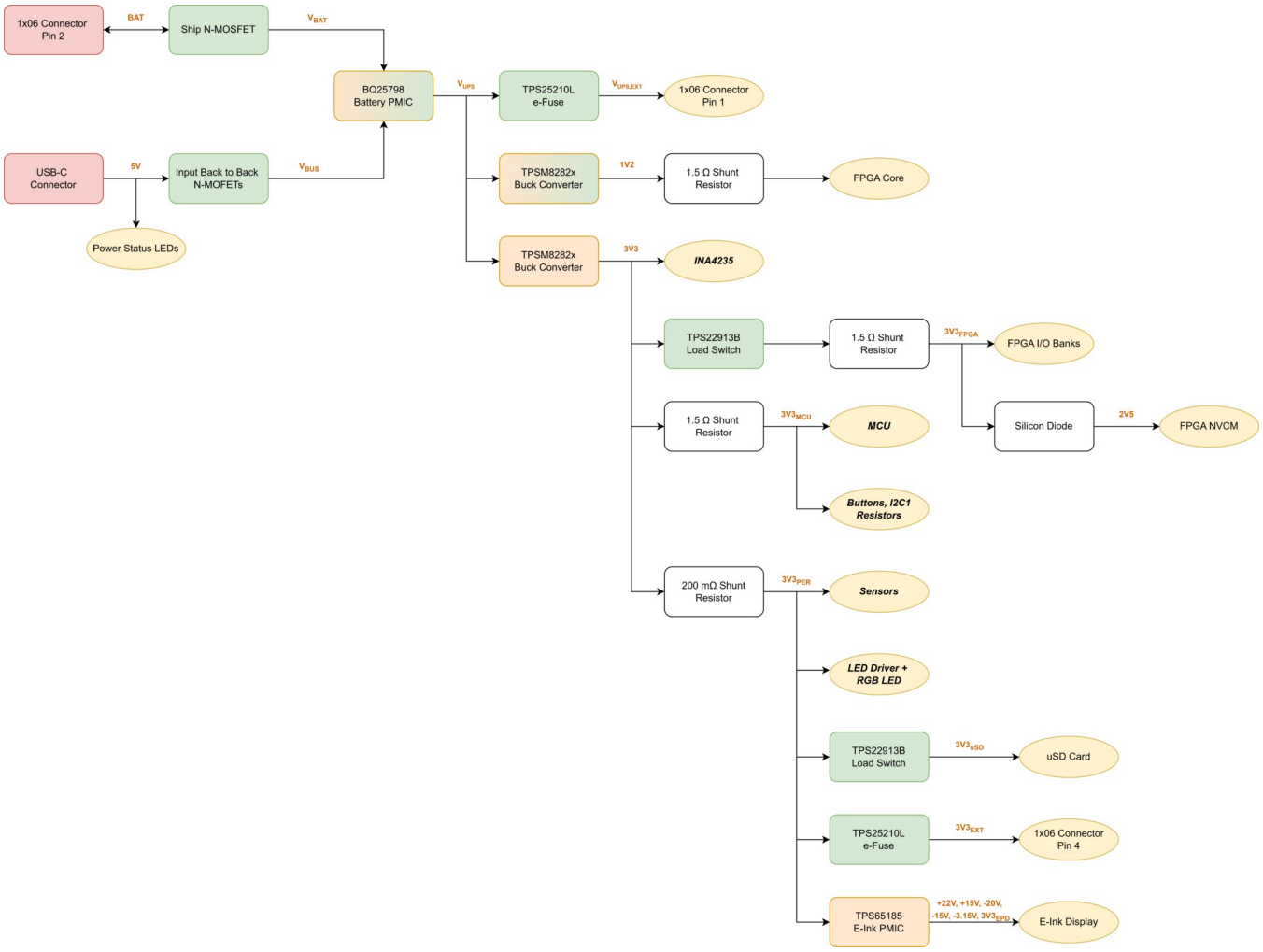
InkLab's power system is among the more sophisticated aspects of the design. It is built around the **Texas Instruments BQ25798** battery charger PMIC, which accepts 5V from a standard USB-C connector. The BQ25798 manages charge/discharge for a Li-Ion battery pack and provides a UPS-style power path, ensuring the system continues running seamlessly if USB power is removed.

USB power passes through a pair of **back-to-back N-MOSFETs** for input protection. A **Ship Mode N-MOSFET** on the battery line allows the charger to be completely disconnected during shipping to prevent drain. A **TPS65185 E-Ink PMIC** generates the multiple voltage rails required by the e-paper panel.

5.2 Power Distribution Tree

Refer to the power tree diagram on the following page. The diagram shows the complete power distribution network. Key characteristics:

- **V_BUS (5V USB input):** Enters through back-to-back N-MOSFETs → BQ25798 PMIC and power status LEDs.
- **V_BAT (battery):** Routed through Ship Mode MOSFET → BQ25798. Battery connector also exposed at 1x06 Pin 2.
- **V_UPS (UPS output from BQ25798):** Feeds two downstream paths: (1) TPS25210L e-Fuse → V_UPS_EXT at 1x06 connector Pin 1; (2) two TPSM8282x synchronous buck converters generating 1.2V and 3.3V.
- **1.2V rail:** Goes through a 1.5Ω shunt resistor (INA monitoring) → FPGA Core power.
- **3.3V main rail:** Monitored by INA4235 at the source, then distributes to: TPS22913B load switch → 3V3_FPGA (FPGA I/O banks, via silicon diode → 2.5V for FPGA NVCM); direct → 3V3_MCU (MCU, buttons, I2C1 resistors); TPS25210L e-Fuse → 3V3_EXT (1x06 Pin 4 external connector); TPS22913B load switch → 3V3_uSD (SD card); and TPS65185 E-Ink PMIC → EPD rails.
- **3V3_PER (peripheral 3.3V via 200mΩ shunt):** Powers: sensors, LED driver + RGB LED, uSD card (through load switch), EXT connector (through e-fuse), and the E-Ink PMIC.



Each power rail includes current monitoring via shunt resistors and INA sensors. The load switches (TPS22913B) and e-fuses (TPS25210L) allow individual rails to be shut down programmatically, either for power saving or as a protection response. The system has extensive protection features: overcurrent, overvoltage, thermal shutdown, and reverse polarity protection at multiple nodes.

6. Software Stack

6.1 RTOS Task Architecture

The firmware is structured around FreeRTOS (CMSIS-RTOS2 API), with independent tasks communicating via message queues, semaphores, and mutexes. All shared peripherals (I2C buses, SPI) are mutex-protected to prevent race conditions.

Task	Responsibility
USB Task	Receives commands over USB CDC, routes to command parser (frontend_api.c), transmits telemetry and responses.
FPGA Task	Manages SPI DMA to/from iCE40, handles bitstream slot switching, clock configuration, and the BMP wrapper SWD commands.
Telemetry Task	Periodically reads INA sensors and BQ25798 registers, formats output (currently JSON, migrating to binary), supports mute/rate control.
Joystick Task	Polls 5-way analog joystick, generates navigation events for the e-paper UI.
LED Task	Drives RGB indicator LED with pattern logic (idle, busy, error, BMP active, standby).
EPD Task	(Planned) Receives framebuffer from MCU rendering engine, transfers to FPGA over SPI for e-paper display sequencing.
Wasm Task	(Planned) Hosts WAMR interpreter, executes .wasm modules loaded from SD card slots.

7. Vue Web Interface — InkLab DevConsole

7.1 Purpose and Philosophy

The **InkLab DevConsole** is a Vue.js web application that runs in the browser and communicates with the device over USB-CDC (WebSerial API). It is the primary interaction layer for researchers — the interface through which hardware is configured, experiments are run, and data is collected, all without writing or flashing any code.

The DevConsole has three operational modes, selectable from the header:

- **Config Mode:** Low-level hardware configuration. FPGA bitstream management, OTA firmware updates, pin multiplexing, energy mode selection. This is the "setup" phase for a new experiment.
- **Debug Mode:** Interactive command interface and automation. The Terminal tab allows raw command execution. The JS Automation tab lets users write JavaScript-like scripts (macros) that execute sequences of commands — the equivalent of a hardware test bench script, but in the browser.
- **App Mode:** The final end-user interface. Hides all developer controls (energy mode, debug button, DevConsole branding). This is what a patient, a test subject, or a product client would see — sensor readings, visualizations, and status information formatted for non-technical users.

7.2 DevConsole Tabs

The following tabs are implemented (Config and Debug sections):

Tab	Function
Terminal	Interactive command-line interface to the device. Live telemetry header showing all power rails, FPGA slot, and connection status. Quick command library with searchable macros.
SPI DMA Tester	FPGA testing and benchmarking. GPIO state viewer, internal BRAM explorer, mixed-signal benchmarks (PWM→ADC, cross-protocol routing), bulk DMA throughput tests.
JS Automation	Script runner for the SD card interpreter. Executes sequences of commands as macros. Benchmarks three execution architectures: Browser JS, SD Macro Interpreter, and Native C.
Programmer	FPGA bitstream upload (drag-and-drop .bin to SD slot), MCU OTA firmware update, FPGA clock divider control, SD card formatting, default bitstream selection.
Power Profiler	Real-time waveform viewer for all four INA-monitored power rails. Trigger modes, CSV/SVG/PNG export, auto-reset on threshold, fast-mode 10-bit acquisition.
Device IO	5-way joystick live vector display, joystick hardware action mapping, RGB LED brightness controls (per-channel), mute control for telemetry output.
Pinout & Routing	Interactive board diagram showing MCU and FPGA pin states. Color-coded by mode (INPUT, OUTPUT, ALTERNATE, ANALOG). Apply routing changes to hardware live. Pin capacitance viewer.
App Mode	Patient/client-facing view. Sensor readings, glucose trend chart, patient profile. All developer UI elements hidden.

8. Extending the Firmware — Developer Guide

8.1 The Four Ways to Run Code on InkLab

InkLab provides four distinct execution paths for running code. Each has different performance characteristics, complexity, and use cases. Choosing the right one depends on what you are trying to accomplish.

Path 1 — Native C Library + Frontend API Command (Best Performance)

This is the standard path for adding permanent, high-performance functionality to the firmware. The workflow is:

4. **Create a new C library:** Write your module as a self-contained .c/.h pair in `Core/Src/` and `Core/Inc/`. Follow the existing module pattern (e.g., battery_soc.c, powerMonitor.c) — init function, a clean public API, no global state leakage.
5. **Add the header to the API file:** Include your new header in `frontend_api.c`.
6. **Create or extend a command class:** In the command parser, either create a new command prefix (e.g., `"MYMODULE:"`) or add commands to an existing class. Command classes group related functionality.
7. **Add the handler logic:** In the command dispatch section, add your `else if (startsWith(cmd, "MYMODULE:DO_THING"))` block. Call your library functions with appropriate input validation and range checks.
8. **Use from the DevConsole Terminal:** Your command is now available immediately over USB-CDC — type it in the Terminal tab or use it in a JS Automation script.

When to use: Time-sensitive operations, high-frequency polling, anything that needs hardware peripherals directly, or functionality that will be used repeatedly. This is zero-overhead — it runs as compiled machine code inside the existing RTOS task.

PERFORMANCE	Native C commands: near-zero overhead. Execution time is bounded by the operation itself (e.g., an I2C read). The USB round-trip for a single command is typically 1–5ms including CDC framing.

Path 2 — Dedicated RTOS Task (Time-Critical Background Loops)

If a new function needs to run continuously in the background — independently of USB commands — it should be its own FreeRTOS task. Example use cases: continuous sensor polling with hardware FIFO integration, real-time PID control loops, or a background logger that writes to SD without blocking the USB task.

Create the task with `osThreadNew()`, assign it a priority appropriate to its timing requirements, and use queues or semaphores to communicate results to other tasks (e.g., push readings to the Telemetry task queue). Be mindful of the stack budget — each task's stack is carved from the 640 KB SRAM.

When to use: Anything that must run at a fixed rate regardless of what the user is doing over USB. Hardware interrupt handlers that need non-trivial follow-up processing. Background data acquisition for logging.

PERFORMANCE	Equivalent to native C — tasks run as compiled code with RTOS preemption. Context switch overhead on Cortex-M33 at 250 MHz is negligible (microseconds). No interpretation penalty.

Path 3 — SD Card Macro Interpreter (Rapid Prototyping & Automation)

InkLab includes a simple text-based interpreter that reads script files from the SD card and executes sequences of frontend API commands. Scripts are plain text files with one command per line. This allows a developer to create complex automation routines, hardware-in-the-loop test sequences, or performance benchmarks without reflashing the MCU.

Scripts can be triggered from the JS Automation tab in the DevConsole, or loaded automatically at boot from a designated slot. The interpreter supports basic control flow (loops, conditionals) and can read back telemetry values to drive conditional logic.

When to use: Automated test sequences (run 50 SPI transactions, verify responses, log results to SD). Repeatable benchmarks. Experiment protocols that need to be shared with collaborators who are not embedded engineers — they can edit a plain-text script file. Quick iteration when you want to test a sequence of commands without writing C.

PERFORMANCE	Moderate overhead compared to native C. Each command goes through text parsing plus the USB-CDC round-trip internally. Suitable for sequences where operations take tens of milliseconds each. Not suitable for microsecond-precision timing.

Path 4 — WebAssembly (WAMR) Sandbox (Hardware-Agnostic Applications)

The planned WAMR integration allows researchers to write application logic in **Rust, C/C++, Python, or Go** (compiled to .wasm), upload the module to an SD card slot, and activate it via a USB command — without touching the MCU firmware. The Wasm module runs in a fully sandboxed linear memory environment with no direct access to MCU registers or RTOS state.

Interaction with hardware is through a registered native API bridge — C functions are exposed to the Wasm module by name (e.g., `read_ina_current(rail_id)`, `write_fpga_reg(addr, val)`). The module cannot crash the system; a fault inside Wasm is caught by the runtime and reported over USB.

When to use: Researchers from non-embedded backgrounds (AI/ML, medical, signal processing) who want to deploy algorithm logic without a cross-compilation toolchain. Multi-tenant scenarios where different researchers each have their own Wasm module. Rapid iteration on application logic without firmware reflash cycles. Safe experimentation in a classroom or lab setting.

PERFORMANCE	Highest overhead of the four paths. WAMR interpreter mode carries a 2–10x slowdown vs. native C depending on the workload. AOT (Ahead-of-Time) compilation reduces this

to ~1.5–2x but requires more Flash. Not suitable for real-time control loops or high-frequency data acquisition. Ideal for application-level logic (data processing, display updates, BLE packet formatting) where the bottleneck is I/O, not CPU.

8.2 Choosing the Right Execution Path

Use Case	Best Path	Example
Need real-time, fixed-rate background loop?	RTOS Task	Continuous sensor FIFO polling, PID control
Need permanent, high-performance hardware control?	Native C + API	New sensor driver, custom SPI protocol
Need automated test sequences or experiment scripts?	SD Macro	Hardware-in-the-loop tests, benchmarks
Need to deploy algorithm logic without reflashing?	WAMR sandbox	AI inference, glucose algorithm, data fusion
Need to automate a sequence from the browser?	JS Automation tab	Multi-step bringup, calibration routines

9. WebAssembly Runtime (WAMR) Integration

9.1 Native Function Bridge

Wasm modules interact with InkLab hardware exclusively through registered C functions. The Wasm module has no direct memory access to MCU address space. Three interaction mechanisms exist:

- **Native function binding:** `wasm_runtime_register_natives()` registers C gateway functions under symbolic names. The Wasm module declares these as imports and calls them normally. Example: `get_ina_power_mW(rail_id)` reads the corresponding INA sensor struct and returns a float.
- **Shared memory via module malloc:** Structured data (telemetry snapshots, image buffers) can be exchanged by allocating a buffer inside Wasm linear memory and using memcpy from the host side. This is the pattern for passing large or structured data.
- **WASI stdout redirect:** WAMR's WASI layer can redirect printf() inside Wasm to USB_Printf(), so a researcher's debug output appears in the DevConsole Terminal automatically.

RAM WARNING

WAMR interpreter build requires ~100–150 KB RAM. Full heap/stack audit required before integration. The 8 KB RX ring buffer and static FATFS workspace are primary reduction candidates. AOT compilation is not recommended on this target due to Flash constraints.

10. RTOS Strategy: FreeRTOS → Zephyr

10.1 Decision Summary

DECISION	Zephyr is the correct long-term target, particularly for its native WAMR integration, formal device tree model (replacing pinmux.c), and MCUmgr OTA. Keep FreeRTOS as the stable v2 release. Port to Zephyr in a parallel branch, starting with the self-contained modules.

10.2 FreeRTOS vs. Zephyr Comparison

Dimension	FreeRTOS + WAMR	Zephyr + WAMR
WAMR integration	Manual port, custom task	Native Kconfig module, tested
Pin multiplexing	Custom pinmux.c	Device tree / pinctrl (formal, portable)
OTA updates	Custom dual-bank logic	MCUmgr — standard tooling
USB CDC	STM32 HAL / CubeMX	Zephyr USB subsystem
Code portability	STM32-specific HAL	Portable across all Zephyr targets
Migration cost	Zero (already running)	Significant — rewrite HAL calls
RAM overhead	Minimal kernel	Slightly higher (~10–20 KB)
Community	Mature, well-documented	Large, fast-growing, complex

10.3 From STM32 HAL to Zephyr API — What Actually Changes

In a Zephyr application, you do not call STM32 HAL functions (`HAL_GPIO_WritePin`, `HAL_I2C_Mem_Read`, etc.). Zephyr provides its own portable abstraction layer, and the STM32 HAL is used internally by Zephyr's drivers — you never touch it directly. This is the single most important mental shift for the port.

Current STM32 HAL Call	Zephyr Replacement
HAL_GPIO_WritePin(port, pin, state)	gpio_pin_set_dt(&gpio_spec, value)
HAL_I2C_Mem_Read(...)	i2c_burst_read_dt(&i2c_dev, reg, buf, len)
osDelay(ms)	k_sleep(K_MSEC(ms))
osThreadNew(...)	K_THREAD_DEFINE(...)
osMutexAcquire(mutex, timeout)	k_mutex_lock(&mutex, K_MSEC(timeout))
HAL_SPI_Transmit_DMA(...)	spi_transceive_dt(&spi_spec, &tx, &rx)
FATFS + f_mount()	Zephyr FS subsystem + CONFIG_FAT_FILESYSTEM_ELM=y

10.4 Setting Up the Zephyr Workspace

Zephyr follows an **application-out-of-tree model**: Zephyr is a dependency of your project, pulled by the `west` meta-tool. You do not develop inside the Zephyr repository.

9. **Install prerequisites:** Python 3.10+, CMake 3.20+, Ninja, west (`pip install west`), and the Zephyr ARM toolchain (arm-zephyr-eabi).
10. **Initialize workspace (once):**

```
mkdir inklab-workspace && cd inklab-workspace
west init -m https://github.com/your-org/inklab-firmware --mr main
west update
```

11. **Project structure:** Your repo (`inklab-firmware/`) contains your code and a `west.yml` that declares the Zephyr version. After `west update`, the workspace contains your app, the full Zephyr tree, and all STM32 HAL shims — all managed by west, none committed to your repo.
12. **VS Code setup:** Install the "nRF Connect for VS Code" or "Zephyr IDE" extension. Both provide GUI build/flash/debug buttons backed by west commands. Abandon STM32CubeIDE for all Zephyr work.

10.5 Port Priority Order

13. **[NOW] Board DTS:** Write inklab_v2.dts while the schematic is current. Pin assignments, I2C buses, SPI, USB, SD card. This is the foundation for every subsequent step.
14. **[NOW] powerMonitor.c → Zephyr:** This module is pure I2C + RTOS mutex. Port it as a learning exercise — swap HAL_I2C_Mem_Read and osMutexAcquire, validate on v1 hardware. No risk, builds muscle memory.
15. **[NOW] Heap/stack audit on v1:** Enable FreeRTOS trace facilities, measure per-task RAM. This data is required before WAMR integration can be planned.
16. **[PARALLEL] USB CDC subsystem:** Enable CONFIG_USB_CDC_ACM=y, replace CDC_Transmit_FS() with uart_fifo_fill() on the CDC device node.
17. **[PARALLEL] FATFS / SD card:** CONFIG_FAT_FILESYSTEM_ELM=y + CONFIG_DISK_DRIVER_SDMMC=y. ELM FatFs calls (f_open, f_read) remain nearly identical.
18. **[ON V2 ARRIVAL] WAMR task:** CONFIG_WASM_MICRO_RUNTIME=y, implement wasm_host.c to load .wasm from SD, register native symbols, run interpreter in k_thread.

11. Related Projects & Prior Art

Project	Relevance to InkLab
Glasgow Interface Explorer	FPGA-based hardware interface tool. Similar lab-on-board concept. Pure FPGA, no MCU. Reference for FPGA-as-instrument architecture.
GreatFET One	NXP LPC4330 with modular expansion boards. Closest analog to the InkLab motherboard/daughterboard concept and frontend API design.
LUNA (Great Scott Gadgets)	FPGA USB analyzer. Clean FPGA ↔ host software interface design.

Project	Relevance to InkLab
	Reference for the BMP debug integration approach.
Black Magic Probe	Direct ancestor of InkLab's embedded SWD core. InkLab wraps BMP in the FPGA, replacing the external probe entirely.
WAMR on Cortex-M0+	Community experiments on RP2040 confirm Wasm on bare-metal RTOS is feasible. InkLab's H562 has significantly more headroom.
Zephyr WASM sample	Upstream Zephyr CONFIG_WASM_MICRO_RUNTIME sample on STM32. Direct reference for the Zephyr port integration.

12. Engineering Notes & Active Watchpoints

12.1 Known Risks

RISK RAM budget is tight. With WAMR (~150KB), FreeRTOS tasks, the 8KB RX ring buffer, FATFS workspace, and per-task stacks, the 640KB budget must be carefully accounted for. Run vTaskGetRunTimeStats() + uxTaskGetSystemState() on v1 hardware before committing to WAMR integration.

RISK .noinit section placement: verify in the STM32H562 linker script that .noinit is outside the BSS zero-init region. CubeMX-generated scripts commonly place it inside BSS, zeroing it on reset and breaking the last_active_slot persistent memory feature.

RISK BQ25798 25ms backup recovery delay: verify whether the latest firmware revision fixes this edge case. Test the USB → battery → USB power path transition under full load to confirm no transient dropout.

NOTE GPIO toggle timer safety floor is 5ms (100Hz max). This is intentional and correct for RTOS health. Document it explicitly in the researcher-facing API so users are not surprised by the ceiling when expecting high-frequency GPIO control.

12.2 Future Enhancements

- **CMSIS-DAP alongside BMP:** Expose CMSIS-DAP from the FPGA SWD core. Enables native VS Code + OpenOCD + pyOCD support without GDB/BMP knowledge, broadening IDE compatibility significantly.
- **MCUmgr OTA protocol:** Replace the custom dual-bank OTA with MCUmgr. Standard tooling (mcumgr-cli, mobile apps), Zephyr-native, and more robust error handling.
- **RISC-V soft core on iCE40UP5K:** The FPGA can host PicoRV32 or SERV. This would give the FPGA its own firmware rather than just RTL, enabling it to act as a true programmable co-processor rather than a fixed-function accelerator.
- **Pub/Sub message bus:** As WAMR joins the task graph, a lightweight topic-based pub/sub layer (on top of FreeRTOS queues or Zephyr's zbus) will let Wasm modules subscribe to telemetry events without polling.

- **Wasm module registry on SD:** Mirror the FPGA slot system for Wasm: named .wasm slots on SD, selectable by USB command. A researcher copies experiment.wasm to the SD card and activates it — no reflash cycle.
- **Binary telemetry protocol:** Migrate from JSON to a binary framing protocol (COBS or similar) to reduce RTOS overhead and increase telemetry throughput. The Terminal tab already has planned support for this migration.

13. v2 PCB Bringup Checklist

19. **[NOW] Heap/stack audit on v1.** Run FreeRTOS trace, measure exact per-task RAM. Baseline required before WAMR planning.
20. **[NOW] Zephyr workspace setup.** Get west + Zephyr building a blinky on the closest STM32H562 board target (nucleo_h562zi). Confirms toolchain before v2 hardware arrives.
21. **[NOW] Board DTS authoring.** Document all v2 pin assignments in inklab_v2.dts while schematic is fresh. This is the highest-leverage preparation task.
22. **[NOW] Port powerMonitor.c to Zephyr.** Self-contained I2C module. Port it, test on v1, confirm the Zephyr I2C API works as expected on STM32H5.
23. **[NOW] WAMR interpreter build test on v1.** Try the WAMR minimal interpreter build on FreeRTOS. Measure actual RAM footprint and execution latency with a trivial Wasm test module.
24. **[NOW] DevConsole backlog items.** The v2 slides identify a long list of UI and firmware improvements (binary protocol, INA4235 phantom fix, 2000-point waveform capture, searchable command library). Work through these on v1 hardware while v2 PCB is being manufactured.
25. **[ON V2 ARRIVAL] Power path validation.** First bringup test: verify the BQ25798 USB→battery→USB transition under load. Monitor all rails with INA sensors during switchover. Confirm no glitches on 3.3V MCU or 1.2V FPGA core.
26. **[ON V2 ARRIVAL] EPD display pipeline test.** Verify FPGA receives SPI framebuffer correctly and generates correct parallel timing for the e-paper panel. Test overdrive protection logic — confirm unchanged pixels are not being driven.
27. **[ON V2 ARRIVAL] BMP/SWD core validation.** Load the SWD RTL bitstream and verify the FPGA can program a known-good SWD target. Measure actual throughput. Target: ≥100 KB/s.

14. Context for AI-Assisted Development Sessions

This section is designed to be copied into the beginning of a new AI conversation to immediately establish full project context. State the current task explicitly after pasting — the AI will have everything it needs to give precise, implementation-level answers without generic advice.

14.1 Quick Reference Card

Parameter	Value
Project name	InkLab — open-source heterogeneous embedded development platform
Primary MCU	STM32H562RIV — Cortex-M33, 250MHz, 2MB Flash (dual-bank),

Parameter	Value
	640KB SRAM
FPGA	Lattice iCE40UP5K — 5280 LUTs — connected via SPI3 with DMA
Display	E-paper (EPD) driven via parallel 8-bit from FPGA — FPGA manages timing and overdrive protection — MCU renders images via LVGL/1bpp
Power PMIC	BQ25798 (5V USB-C input, NOT PD) + INA4235 + 4x INA series monitors — extensive power tree with load switches and e-fuses
E-Ink power	TPS65185 PMIC — generates +22V, +15V, -20V, -15V, -3.15V, 3V3_EPD from main 3.3V rail
RTOS	FreeRTOS (CMSIS-RTOS2) — migrating to Zephyr — tasks: USB, FPGA, Telemetry, Joystick, LED, EPD (planned), Wasm (planned)
USB interface	Full-speed CDC — JSON command protocol (migrating to binary) — 8KB RX ring buffer
Storage	SD card via SPI + FATFS — up to 16 FPGA bitstream slots (configurable constant) + Wasm module slots planned
Web interface	Vue.js DevConsole — WebSerial API over USB-CDC — Config / Debug / App modes
Debug core	BMP wrapper on MCU (RTOS task) → SPI commands to FPGA SWD RTL module (already built) — targets ≥100 KB/s programming
Wasm runtime	WAMR planned — native function binding model — SD card slot registry — dedicated FreeRTOS/Zephyr task
FPGA toolchain	OSS CAD Suite (Yosys + nextpnr + IceStorm) — fully open source — Verilog
v1 MCU (legacy)	STM32G0B1CEUT6 — M0+, 64MHz, 256KB Flash, 144KB RAM — replaced due to RAM/FPU/IoT limitations
Key source files	app_freertos.c (task init), frontend_api.c (command parser), powerMonitor.c (INA driver), bq25798.c, fpga.c (SPI/DMA), pinmux.c, battery_soc.c, sys_power.c, led_manager.c

End of Document — InkLab Project Reference | v2 Design Phase